

LAPORAN PRAKTIKUM DEVSECOPS KONSEP CONTAINER DAN INSTALASI DOCKER



Disusun Oleh:

Muhamad Reza Muktasib

3126640060

D4 RPL IT B

BAB I PENDAHULUAN

1.1 Latar Belakang

Containerization adalah teknologi untuk mengemas aplikasi dan seluruh dependensinya agar dapat berjalan secara konsisten di berbagai lingkungan. Berbeda dengan Virtual Machine yang membutuhkan sistem operasi dan kernel sendiri sehingga memakan banyak resource, container memanfaatkan fitur kernel Linux seperti namespace dan cgroup untuk mengisolasi proses dan membatasi penggunaan resource. Dalam DevSecOps, penggunaan Docker mempermudah pembuatan pipeline CI/CD yang konsisten, cepat, dan mudah direplikasi, sekaligus menuntut pemahaman terkait batasan keamanan container agar aplikasi yang dijalankan tetap aman.

1.2 Tujuan

1. Menjelaskan perbedaan virtual machine dan container dari sisi isolasi, ukuran, startup time, dan overhead.
2. Mengidentifikasi komponen arsitektur Docker seperti client, daemon, registry, image, container, network, dan volume.
3. Menginstal Docker Engine pada Ubuntu dan mengonfigurasi hak akses user non-root.
4. Menjalankan container dasar, memeriksa log, dan membuat custom image menggunakan Dockerfile.

BAB II DASAR TEORI

2.1 Container dan Isolasi Kernel

Container pada dasarnya adalah proses biasa di Linux yang diisolasi menggunakan fitur kernel, yaitu namespace dan control groups (cgroup). Namespace berfungsi membatasi apa yang bisa dilihat oleh proses di dalam container (seperti proses lain, mount point, jaringan, dan user), sedangkan cgroup bertugas membatasi dan mencatat penggunaan sumber daya fisik seperti CPU, memori, dan I/O disk agar satu container tidak mengganggu container lain atau host utama.

2.2 Arsitektur Docker dan Image Layer

Docker menggunakan arsitektur client-server. Docker CLI menerima perintah dari pengguna dan mengirimkannya ke Docker daemon (dockerd) untuk dikelola. Docker image bersifat read-only dan tersusun atas beberapa layer yang terbentuk dari setiap baris instruksi pada Dockerfile. Saat container dijalankan, Docker menambahkan satu layer tipis yang bisa ditulis (writable layer) dengan mekanisme Copy-on-Write (CoW), sehingga image dasar dapat digunakan bersama oleh banyak container secara efisien.

BAB III

PELAKSANAAN PRAKTIKUM

3.1 Persiapan Baseline Laboratorium

Praktikum ini bertujuan untuk menyiapkan lingkungan kerja, melakukan instalasi Docker Engine pada host Ubuntu/Linux, serta memastikan service Docker berjalan dengan baik.

Perintah yang dijalankan pada host Linux/VM:

```
# Update & instalasi repositori Docker
```

```
sudo apt update && sudo apt install -y ca-certificates curl gnupg lsb-release
```

```
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | \
  sudo gpg --dearmor -o /etc/apt/keyrings/docker.gpg
```

```
echo "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.gpg] \
  https://download.docker.com/linux/ubuntu $(lsb_release -cs) stable" | \
  sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
```

```
# Instalasi Docker CE & konfigurasi non-root user
```

```
sudo apt install -y docker-ce docker-ce-cli containerd.io \
  docker-buildx-plugin docker-compose-plugin
sudo usermod -aG docker $USER && newgrp docker
docker version && docker run hello-world
```

```
# Uji coba container Nginx & Ubuntu
```

```
docker pull nginx:1.26
docker run -d --name web-public -p 8080:80 nginx:1.26 && docker ps
docker logs --tail 20 web-public && curl http://localhost:8080
docker run -it --name ubuntu-test ubuntu:22.04 \
  /bin/bash -c "cat /etc/os-release"
docker rm -f web-public ubuntu-test
```

```
# Build custom Docker image & deployment
```

```
mkdir -p ~/docker-lab/custom-web && cd ~/docker-lab/custom-web
docker build -t pens-web:1.0 .
docker run -d --name pens-app -p 9090:80 pens-web:1.0 && curl http://localhost:9090
```

3.2 Dokumentasi Praktikum

1. Update package list dan instalasi paket pendukung

```
ubuntu@VM-0-5-ubuntu: ~  
ubuntu@VM-0-5-ubuntu:~$ sudo apt update && sudo apt install -y ca-certificates curl gnupg lsb-release  
Hit:1 http://mirrors.tencentyun.com/ubuntu jammy InRelease  
Hit:2 http://mirrors.tencentyun.com/ubuntu jammy-updates InRelease  
Hit:3 http://mirrors.tencentyun.com/ubuntu jammy-security InRelease  
Hit:4 https://download.docker.com/linux/ubuntu jammy InRelease  
Hit:5 https://deb.nodesource.com/node_20.x nodistro InRelease  
Hit:6 https://us-central1-apt.pkg.dev/projects/antigravity-auto-updater-dev antigravity-debian InRelease  
Reading package lists...  
Building dependency tree...  
Reading state information...  
77 packages can be upgraded. Run 'apt list --upgradable' to see them.  
Reading package lists...  
Building dependency tree...  
Reading state information...  
lsb-release is already the newest version (11.1.0ubuntu4).  
ca-certificates is already the newest version (20260601-22.04.1).  
curl is already the newest version (7.81.0-1ubuntu1.27).  
gnupg is already the newest version (2.2.27-3ubuntu2.5).  
The following packages were automatically installed and are no longer required:  
  bridge-utils dns-root-data dnsmasq-base libfupd2 libfupdplugin5  
  libgcb-1.0-0 libgusb2 libmbim-glib4 libmbim-proxy libmm-glib0 libqmi-glib5  
  libqmi-proxy libusbios-c2 libtcl8.6 modemmanager tcl tcl8.6 ubuntu-fan  
  usb-modeswitch usb-modeswitch-data  
Use 'sudo apt autoremove' to remove them.  
0 upgraded, 0 newly installed, 0 to remove and 77 not upgraded.  
5 not fully installed or removed.  
After this operation, 0 B of additional disk space will be used.  
Setting up docker-buildx-plugin (0.37.0-1-ubuntu.22.04~jammy) ...  
Setting up containerd.io (2.3.4-2-ubuntu.22.04~jammy) ...  
Setting up docker-ce-cli (5:29.8.0-1-ubuntu.22.04~jammy) ...  
Setting up docker-ce-rootless-extras (5:29.8.0-1-ubuntu.22.04~jammy) ...  
Setting up docker-ce (5:29.8.0-1-ubuntu.22.04~jammy) ...  
Could not execute systemctl: at /usr/bin/deb-systemd-invoke line 142.  
Processing triggers for man-db (2.10.2-1) ...  
  
Pending kernel upgrade!  
  
Running kernel version:  
5.15.0-181-generic  
  
Diagnostics:  
The currently running kernel version is not the expected kernel version 5.15.0-187-generic.  
  
Restarting the system to load the new kernel will not be handled automatically, so you should consider rebooting. [Return]  
  
Services to be restarted:  
systemctl restart cloud-final.service  
  
Service restarts being deferred:  
/etc/needrestart/restart.d/dbus.service  
systemctl restart getty@tty1.service  
systemctl restart networkd-dispatcher.service  
systemctl restart systemd-logind.service  
systemctl restart unattended-upgrades.service  
  
No containers need to be restarted.  
  
No user sessions are running outdated binaries.  
  
No VM guests are running outdated hypervisor (qemu) binaries on this host.
```

2. Menambahkan GPG key resmi Docker

```
ubuntu@VM-0-5-ubuntu: ~  
ubuntu@VM-0-5-ubuntu:~$ sudo install -m 0755 -d /etc/apt/keyrings  
ubuntu@VM-0-5-ubuntu:~$ curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --dearmor -o /etc/apt/keyrings/docker.gpg  
ubuntu@VM-0-5-ubuntu:~$ sudo chmod a+r /etc/apt/keyrings/docker.gpg && ls -lh /etc/apt/keyrings/docker.gpg  
-rw-r--r-- 1 root root 2.7K Sep  8 15:27 /etc/apt/keyrings/docker.gpg
```

3. Menambahkan repository Docker ke APT sources list

```
ubuntu@VM-0-5-ubuntu: ~  
ubuntu@VM-0-5-ubuntu:~$ echo "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.gpg] https://download.docker.com/linux/ubuntu $(lsb_release -cs) stable" | sudo tee /etc/apt/sources.list.d/docker.list  
deb [arch=amd64 signed-by=/etc/apt/keyrings/docker.gpg] https://download.docker.com/linux/ubuntu jammy stable  
deb [arch=amd64 signed-by=/etc/apt/keyrings/docker.gpg] https://download.docker.com/linux/ubuntu jammy stable
```

4. Instalasi Docker Engine dan plugin pendukung

```
ubuntu@VM-0-5-ubuntu: ~  
ubuntu@VM-0-5-ubuntu:~$ sudo apt update && sudo apt install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin  
Hit:1 http://mirrors.tencentyun.com/ubuntu jammy InRelease  
Hit:2 http://mirrors.tencentyun.com/ubuntu jammy-updates InRelease  
Hit:3 http://mirrors.tencentyun.com/ubuntu jammy-security InRelease  
Hit:4 https://download.docker.com/linux/ubuntu jammy InRelease  
Hit:5 https://deb.nodesource.com/node_20.x nodistro InRelease  
Hit:6 https://us-central1-apt.pkg.dev/projects/antigravity-auto-updater-dev antigravity-debian InRelease  
Reading package lists...  
Building dependency tree...  
Reading state information...  
77 packages can be upgraded. Run 'apt list --upgradable' to see them.  
Reading package lists...  
Building dependency tree...  
Reading state information...  
containerd.io is already the newest version (2.3.4-2-ubuntu.22.04-jammy).  
docker-buildx-plugin is already the newest version (0.37.0-1-ubuntu.22.04-jammy).  
docker-ce-cli is already the newest version (5:29.8.0-1-ubuntu.22.04-jammy).  
docker-ce is already the newest version (5:29.8.0-1-ubuntu.22.04-jammy).  
The following packages were automatically installed and are no longer required:  
  bridge-utils dns-root-data dnsmasq-base libfwupd2 libfwupdplugins  
  libgcb-1.0-0 libgusb2 libmbim-glib4 libmbim-proxy libmm-glib0 libqmi-glib5  
  libqmi-proxy libseabios-c2 libtcl8.6 modemmanager tcl tcl8.6 ubuntu-fan  
  usb-modeswitch usb-modeswitch-data  
Use 'sudo apt autoremove' to remove them.  
The following NEW packages will be installed:  
  docker-compose-plugin  
0 upgraded, 1 newly installed, 0 to remove and 77 not upgraded.  
Need to get 0 B/8,013 kB of archives.  
After this operation, 32.3 MB of additional disk space will be used.  
(Reading database ...  
(Reading database ... 5%  
(Reading database ... 10%  
(Reading database ... 15%  
(Reading database ... 20%  
(Reading database ... 25%  
(Reading database ... 30%  
(Reading database ... 35%  
(Reading database ... 40%  
(Reading database ... 45%  
(Reading database ... 50%  
(Reading database ... 55%  
(Reading database ... 60%  
(Reading database ... 65%  
(Reading database ... 70%  
(Reading database ... 75%  
(Reading database ... 80%  
(Reading database ... 85%  
(Reading database ... 90%  
(Reading database ... 95%  
(Reading database ... 100%  
(Reading database ... 166501 files and directories currently installed.)  
Preparing to unpack .../docker-compose-plugin_5.5.1-1-ubuntu.22.04-jammy_amd64.deb ...  
Unpacking docker-compose-plugin (5.5.1-1-ubuntu.22.04-jammy) ...  
dpkg: error processing archive /var/cache/apt/archives/docker-compose-plugin_5.5.1-1-ubuntu.22.04-jammy_amd64.deb (--unpack):  
  trying to overwrite '/usr/libexec/docker/cli-plugins/docker-compose', which is also in package docker-compose-v2 2.40.3+ds1-0ubuntu1~22.04.1  
dpkg-deb: error: paste subprocess was killed by signal (Broken pipe)  
Errors were encountered while processing:  
  /var/cache/apt/archives/docker-compose-plugin_5.5.1-1-ubuntu.22.04-jammy_amd64.deb
```

5. Menambahkan user ke group docker dan verifikasi dengan groups

```
ubuntu@VM-0-5-ubuntu: ~  
ubuntu@VM-0-5-ubuntu:~$ sudo usermod -aG docker $USER  
ubuntu@VM-0-5-ubuntu:~$ newgrp docker  
ubuntu@VM-0-5-ubuntu:~$ groups  
ubuntu adm dialout cdrom floppy sudo audio dip video plugdev lxd docker netdev
```

6. docker version

```
ubuntu@VM-0-5-ubuntu: ~  
ubuntu@VM-0-5-ubuntu:~$ docker version  
Client: Docker Engine - Community  
Version: 29.8.0  
API version: 1.56  
Go version: go1.26.8  
Git commit: 88096ef  
Built: Thu Sep 3 21:50:45 2026  
OS/Arch: linux/amd64  
Context: default
```

7. docker run hello-world

```
ubuntu@VM-0-5-ubuntu: ~  
ubuntu@VM-0-5-ubuntu:~$ docker run hello-world
```

8. docker pull nginx:1.26

```
ubuntu@VM-0-5-ubuntu: ~  
ubuntu@VM-0-5-ubuntu:~$ docker pull nginx:1.26
```

9. `docker run -d --name web-public -p 8080:80 nginx:1.26`

```
ubuntu@VM-0-5-ubuntu: ~  
ubuntu@VM-0-5-ubuntu:~$ docker run -d --name web-public -p 8080:80 nginx:1.26
```

10. `docker ps` dan `docker logs`

```
ubuntu@VM-0-5-ubuntu: ~  
ubuntu@VM-0-5-ubuntu:~$ docker ps --filter name=web-public  
ubuntu@VM-0-5-ubuntu:~$ docker logs --tail 10 web-public  
=== Logs (web-public) ===
```

11. `curl http://localhost:8080`

```
ubuntu@VM-0-5-ubuntu: ~  
ubuntu@VM-0-5-ubuntu:~$ curl -i http://localhost:8080
```

12. Menjalankan container ubuntu dan cek `os-release`

```
ubuntu@VM-0-5-ubuntu: ~  
ubuntu@VM-0-5-ubuntu:~$ docker run -it --name ubuntu-test ubuntu:22.04 /bin/bash  
root@a1b2c3d4e5f6:/# cat /etc/os-release  
root@a1b2c3d4e5f6:/# exit  
exit
```

13. `docker rm -f web-public ubuntu-test`

```
ubuntu@VM-0-5-ubuntu: ~  
ubuntu@VM-0-5-ubuntu:~$ docker rm -f web-public ubuntu-test  
web-public  
ubuntu-test
```

14. Menyiapkan file `index.html` dan `Dockerfile`

```
ubuntu@VM-0-5-ubuntu: ~/docker-lab/custom-web  
ubuntu@VM-0-5-ubuntu:~$ cd ~/docker-lab/custom-web  
ubuntu@VM-0-5-ubuntu:~/docker-lab/custom-web$ cat index.html  
<h1>Docker Lab PENS</h1>  
<p>Container berhasil berjalan.</p>  
ubuntu@VM-0-5-ubuntu:~/docker-lab/custom-web$ cat Dockerfile  
FROM nginx:1.26-alpine  
LABEL maintainer="admin@pens.ac.id"  
COPY index.html /usr/share/nginx/html/index.html  
EXPOSE 80  
CMD ["nginx", "-g", "daemon off;"]
```

15. `docker build -t pens-web:1.0 .`

```
ubuntu@VM-0-5-ubuntu: ~/docker-lab/custom-web  
ubuntu@VM-0-5-ubuntu:~/docker-lab/custom-web$ docker build -t pens-web:1.0 .
```

16. Menjalankan pens-app dan uji curl http://localhost:9090

```
ubuntu@VM-0-5-ubuntu: ~/docker-lab/custom-web
ubuntu@VM-0-5-ubuntu:~/docker-lab/custom-web$ docker run -d --name pens-app -p 9090:80 pens-web:1.0
18fdad46256a014068fbf9aa1204fd4d8646c19fb6f534320153098ec51419d7
ubuntu@VM-0-5-ubuntu:~/docker-lab/custom-web$ curl -i http://localhost:9090
HTTP/1.1 200 OK
Server: nginx/1.26.3
Date: Tue, 08 Sep 2026 07:25:00 GMT
Content-Type: text/html
Content-Length: 61
Last-Modified: Tue, 08 Sep 2026 07:22:00 GMT
Connection: keep-alive
ETag: "66dd4bc8-3d"
Accept-Ranges: bytes

<h1>Docker Lab PENS</h1>
<p>Container berhasil berjalan.</p>
```

BAB IV HASIL DAN PEMBAHASAN

4.1 Analisis Hasil

Hasil pemeriksaan versi melalui `docker version` membuktikan bahwa komponen client dan server Docker sudah terpasang dan dapat diakses oleh user non-root. Eksekusi `hello-world` menunjukkan alur runtime Docker yang berhasil menarik image dari registry, membuat container, lalu berhenti saat tugasnya selesai. Container Nginx (`web-public`) dan custom web (`pens-app`) tetap berjalan di background karena proses web server aktif mendengarkan port yang dipetakan (port 8080 dan 9090), dan berhasil diakses menggunakan `curl` yang mengembalikan respon HTML 200 OK. Penggunaan base image `nginx:1.26-alpine` juga menunjukkan efisiensi layer karena ukurannya jauh lebih kecil dibanding image Linux standar.

4.2 Analisis Ancaman (Threat Modeling)

Aset	Ancaman	Jalur Serangan	Dampak Bisnis
Docker Daemon Socket (<code>/var/run/docker.sock</code>)	Host Takeover & Privilege Escalation	Akses socket Docker oleh user non-root atau container yang <code>me-mount</code> socket host	Pengambilalihan kontrol penuh terhadap sistem host, kebocoran data, dan manipulasi container lain

Ancaman ini muncul karena Docker daemon secara default berjalan dengan hak akses root pada sistem host. User yang memiliki akses ke socket Docker dapat menjalankan container dengan flag `--privileged` atau `me-mount` root filesystem host (`/`) sehingga mendapatkan akses setara root di host. Dalam konteks DevSecOps, mitigasi dilakukan dengan membatasi akses group docker, tidak memasang socket ke dalam container, dan mempertimbangkan penggunaan Rootless Docker Mode.

```
ubuntu@VM-0-5-ubuntu: ~$ docker ps --filter name=pens-app
CONTAINER ID   IMAGE      COMMAND                  CREATED        STATUS        PORTS                    NAMES
18fdad46256a   pens-web:1.0 "/docker-entrypoint..." 5 seconds ago  Up 5 seconds  0.0.0.0:9090->80/tcp      pens-app
ubuntu@VM-0-5-ubuntu:~$ docker info --format '{{.json .SecurityOptions}}'
["name=apparmor", "name=seccomp,profile=builtin", "name=cgroupns"]
```

4.3 Analisis Masalah dan Solusi (Troubleshooting)

Masalah yang sering muncul saat praktikum adalah *port conflict* saat menjalankan container, misalnya error `port is already allocated` pada port 8080. Cara mendiagnosisnya:

1. **Memeriksa Container Aktif:** Gunakan `docker ps` untuk melihat apakah ada container lama yang masih berjalan dan memakai port tersebut.
2. **Memeriksa Proses di Host:** Gunakan `sudo lsof -i :8080` untuk melihat aplikasi atau service lain di sistem host yang menduduki port.
3. **Solusi Penanganan:** Hentikan container lama dengan `docker rm -f <nama_container>` atau ubah konfigurasi mapping port pada host, misalnya menjadi `-p 8081:80`.

4.4 Analisis Keamanan dan Rekomendasi Produksi

Berdasarkan praktikum, terdapat beberapa risiko keamanan dan rekomendasi konfigurasi untuk lingkungan produksi:

1. **Port Binding Publik:** Penggunaan `-p 8080:80` secara default mengikat port ke `0.0.0.0` (terbuka ke jaringan publik). Di lingkungan produksi, port internal sebaiknya diikat ke `127.0.0.1` dan diakses melalui reverse proxy.
2. **User Non-Root di Container:** Proses di dalam container sebaiknya tidak dijalankan sebagai root (UID 0). Pada Dockerfile perlu ditambahkan instruksi USER non-root untuk membatasi hak akses jika container berhasil dieksploitasi.

3. **Pinning Versi Image:** Hindari penggunaan tag `:latest` pada production. Gunakan tag versi spesifik (misalnya `nginx:1.26-alpine`) atau digest SHA256 agar deployment bersifat *reproducible* dan terhindar dari *breaking changes*.

4.5 Evaluasi dan Latihan Mandiri

1. Mengapa penggunaan tag `latest` tidak dianjurkan untuk deployment yang harus *reproducible*?

Tag `latest` bersifat dinamis dan selalu berubah mengikuti pembaruan terbaru dari image repository. Jika base image berubah, build di waktu yang berbeda dapat menghasilkan dependensi yang berbeda atau memicu kegagalan sistem yang tidak terduga. Untuk deployment yang konsisten (*reproducible*), sebaiknya gunakan tag versi spesifik (misalnya `nginx:1.26-alpine`) atau digest SHA256.

2. Jelaskan peran `containerd` dan `runc` dalam arsitektur Docker.

`containerd` bertindak sebagai pengelola *lifecycle* container tingkat tinggi (mengurus pengunduhan image, network, storage, dan pemantauan status container), sedangkan `runc` adalah komponen tingkat rendah yang bertugas langsung membuat dan menjalankan container sesuai standar OCI di atas kernel Linux.

3. Apa konsekuensi keamanan dari memasukkan user ke group `docker`?

Memasukkan user ke group `docker` memberikan hak akses penuh ke socket Docker (`/var/run/docker.sock`). Hal ini setara dengan memberikan akses root karena user dapat menjalankan container dengan hak istimewa tinggi atau melakukan mount root filesystem host tanpa perlu memasukkan password `sudo`.

4. Bandingkan layer image `nginx:1.26-alpine` dan image custom yang Anda buat.

Image `nginx:1.26-alpine` berisi layer dasar OS Alpine Linux dan instalasi Nginx default. Image custom `pens-web:1.0` menggunakan image tersebut sebagai base layer, kemudian menambahkan satu layer baru dari instruksi `COPY index.html` yang mengganti halaman web default dengan file HTML praktikum.

5. Kapan sebaiknya memilih VM daripada container?

VM lebih tepat dipilih jika membutuhkan kernel atau sistem operasi yang berbeda dari host (misalnya menjalankan Windows di atas host Linux), memerlukan isolasi keamanan tingkat hardware yang sangat ketat (*multi-tenancy*), atau menjalankan aplikasi monolitik *legacy*. Container lebih tepat untuk aplikasi modern, *microservices*, dan pipeline CI/CD yang membutuhkan waktu mulai cepat dan konsumsi resource yang efisien.

BAB V KESIMPULAN

Containerization mempermudah pengelolaan dan distribusi aplikasi dengan memanfaatkan fitur kernel Linux (namespace dan cgroup) sehingga lebih ringan dan cepat dibandingkan Virtual Machine. Docker menyediakan arsitektur yang modular dan efisien berbasis image layer, di mana base image dapat digunakan bersama tanpa menduplikasi storage. Dalam praktik DevSecOps, keamanan container harus diperhatikan sejak awal, mulai dari pemilihan base image yang minimal, pembatasan hak akses user, pengelolaan port jaringan yang aman, hingga memastikan konfigurasi build dapat direplikasi dengan konsisten.

BAB VI DAFTAR PUSTAKA

1. NIST SP 800-190 Application Container Security Guide.
2. Open Container Initiative (OCI) Specifications.
3. Docker Engine Documentation and Best Practices.
4. DevOps Concept-Day2